DOP-C02 · 內容領域 2

組態管理與 IaC

2.1 以 IaC 定義可重複使用的基礎設施
2.2 多帳戶環境自動化 · 2.3 大規模自動化

17%

考試計分占比

依 AWS 官方考試指南（DOP-C02）重新整理

領域 2 總覽：三項任務陳述

2.1

定義雲端基礎設施與可重複使用元件，管理系統整個生命週期

IaC 選項與工具 · IaC 變更管理 · SAM / CloudFormation / CDK · StackSets · 組態管理服務 · Service Catalog 與模組

2.2

自動化建立、納管與保護多帳戶 / 多區域環境中的 AWS 帳戶

帳戶結構最佳實務 · 帳戶佈建自動化 · Organizations / Control Tower · SCP 與角色 · 大規模治理與安全控制

2.3

為複雜任務與大規模環境設計自動化解決方案

清查、組態與修補自動化 · Lambda / SDK / Step Functions · 期望狀態組態 · 維持軟體合規

AWS 上的 IaC 選項與工具

AWS CloudFormation

以 JSON / YAML 宣告資源，AWS 管理堆疊狀態、相依順序與回滾

其他工具的共同底層

AWS SAM

CloudFormation 的無伺服器擴充，以簡潔語法定義 Lambda、API、DynamoDB

Transform 轉換為 CFN

AWS CDK

以 TypeScript、Python、Java 等程式語言定義基礎設施，合成為 CFN 範本

cdk synth → 範本

Terraform / Pulumi

第三方多雲 IaC 工具，自行管理狀態檔（常存 S3 + DynamoDB 鎖定）

Control Tower AFT 使用 Terraform

AWS CLI / SDK

以指令或程式直接呼叫 API；適合腳本與自動化，但不追蹤期望狀態

命令式，非宣告式

AWS Elastic Beanstalk

PaaS：上傳程式碼即建立環境，以 .ebextensions 自訂設定

底層同樣是 CFN 堆疊

宣告式（描述期望狀態，工具負責達成）vs 命令式（描述執行步驟）：考試情境多偏好宣告式 IaC，以便版本控制、審查與重複佈建

CloudFormation 範本結構與關鍵功能

範本區段

- **Parameters** — 部署時輸入的值
- **Mappings** — 依區域 / 環境查表
- **Conditions** — 條件式建立資源
- **Transform** — 巨集，如 SAM
- **Resources** — 唯一必要區段
- **Outputs** — 輸出值、跨堆疊匯出

內建函數

Ref、Fn::GetAtt、Fn::Sub、Fn::Join、Fn::If；Fn::ImportValue 引用其他堆疊匯出值

巢狀堆疊

AWS::CloudFormation::Stack 重複使用共用元件；跨堆疊引用則用 Export / ImportValue

CreationPolicy / cfn-signal

等待執行個體完成啟動腳本並回報成功，才把資源標為完成

UpdatePolicy

控制 Auto Scaling 群組滾動或替換更新的方式

DeletionPolicy

Retain 保留、Snapshot 快照；UpdateReplacePolicy 處理替換時的舊資源

自訂資源

以 Lambda 支援 CFN 原生不支援的動作，需回傳成功 / 失敗

撰寫與部署 IaC：AWS SAM 與 AWS CDK

AWS SAM

- Transform: AWS::Serverless-2016-10-31
- 資源類型：Serverless::Function、Api、SimpleTable、StateMachine
- sam build → sam deploy（產生變更集）
- sam local 在本機測試 Lambda 與 API
- DeploymentPreference：整合 CodeDeploy 做金絲雀 / 線性部署

AWS CDK

- App → Stack → Construct 的階層結構
- Construct 分層：L1（Cfn 資源）、L2（含預設值）、L3（模式）
- cdk bootstrap 先在帳戶 / 區域建立部署資源
- cdk diff 預覽變更、cdk deploy 部署
- Aspects 與 cdk-nag 在合成時檢查安全規則

程式碼 / 範本
Git 版本控制

→

合成 / 轉換
cdk synth / SAM Transform

→

CFN 範本
可審查的產物

→

變更集
預覽新增、修改、替換

→

堆疊
建立或更新資源

IaC 平台的變更管理流程

PR 與審查

範本與參數存於 Git，以 PR 審查

靜態檢查

cfn-lint、cfn-guard、cdk-nag 檢查語法與政策

變更集預覽

了解哪些資源會被修改或替換

手動核准

管道核准關卡，由負責人確認

執行變更

以管道部署，不在主控台手動修改

漂移偵測

找出實際狀態與範本的差異

堆疊政策 Stack policy

防止更新時意外修改或替換關鍵資源（如生產資料庫）

終止保護

啟用後堆疊無法被刪除，需先關閉保護

回滾觸發條件

更新期間 CloudWatch 警示觸發即自動回滾

CloudFormation Hooks 與 Guard 可在建立資源前主動攔截不合規設定（例如未加密的 S3 儲存貯體），屬於「主動式」控制

以 CloudFormation StackSets 跨帳戶與區域部署

管理帳戶 / 委派管理員

一個 StackSet = 一份範本 + 參數

堆疊執行個體（帳戶 × 區域）

	us-east-1	eu-west-1	ap-northeast-1
帳戶 A	堆疊	堆疊	堆疊
帳戶 B	堆疊	堆疊	堆疊
帳戶 C	堆疊	堆疊	堆疊

自行管理的權限

- 管理帳戶：AWSCloudFormationStackSetAdministrationRole
- 目標帳戶：AWSCloudFormationStackSetExecutionRole
- 可部署到不在 Organizations 中的帳戶
- 需自行在每個目標帳戶建立角色

服務管理的權限（Organizations）

- 以 OU 為目標，自動建立所需角色
- 自動部署：新帳戶加入 OU 時自動建立堆疊
- 可註冊委派管理員帳戶執行 StackSets
- 部署選項：最大並行數、容錯數、區域依序 / 平行

選擇最合適的組態管理服務

服務	主要用途	何時選擇
AWS Systems Manager	管理 EC2 與本地伺服器：執行指令、修補、維持期望狀態、參數	需要大規模管理作業系統與軟體組態
AWS Config	記錄資源組態與變更歷史，以規則評估合規	需要稽核「資源設定是否合規」並自動修正
AWS AppConfig	管理應用程式組態與功能旗標，安全漸進地部署	不重新部署程式就要變更設定；需驗證器與自動回滾
AWS OpsWorks	受管 Chef / Puppet（已於 2024 年 5 月終止服務）	考綱仍列出；現行做法改用 Systems Manager



作業系統 / 軟體層

Systems Manager



AWS 資源設定層

AWS Config



應用程式設定層

AWS AppConfig

把模式、治理與安全標準做成可重複使用的 IaC



AWS Service Catalog

管理員把核准的 CloudFormation 範本發布為「產品」，放入「產品組合」，使用者自助佈建

啟動限制：以指定角色佈建，使用者不需資源權限



CloudFormation 模組

把常用資源組合封裝為登錄檔中的模組（類型 ::MODULE），在範本中像資源一樣引用

統一標準設定，集中更新



CDK Construct 程式庫

把安全預設值（加密、記錄、標籤）寫進 L3 Construct，以套件分享給團隊

可透過 CodeArtifact 發布

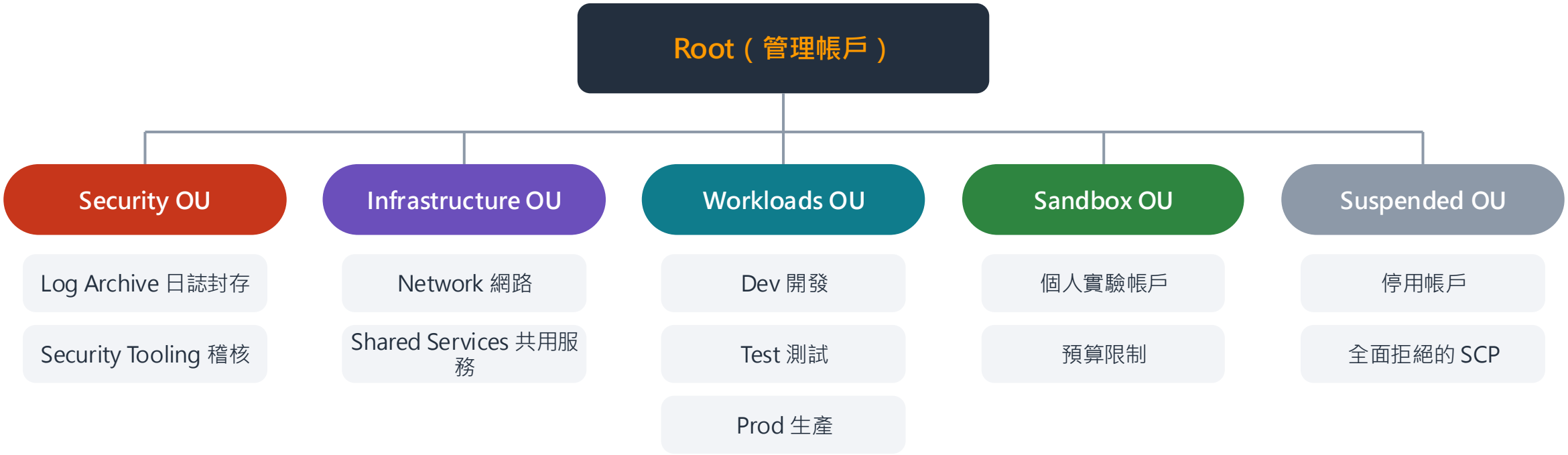


Hooks / Guard / cdk-nag

以政策即程式碼檢查範本，在部署前阻擋不合規資源

主動式治理控制

AWS 多帳戶結構最佳實務



以帳戶為隔離邊界

帳戶是安全、帳單與服務配額的硬邊界

OU 依功能而非組織圖

依安全需求與工作負載類型分組，便於套用 SCP

管理帳戶不放工作負載

SCP 不影響管理帳戶，應僅用於治理與帳單

集中管理帳戶：AWS Organizations 與 Control Tower



AWS Organizations

- 管理帳戶 + 組織單位（OU）階層
- 以 API 建立帳戶或邀請既有帳戶
- 合併帳單與用量折扣共享
- 授權政策：SCP、RCP（資源控制政策）
- 管理政策：標籤政策、備份政策、AI 退出政策
- 委派管理員：讓安全帳戶管理 GuardDuty、Security Hub 等
- 與 CloudTrail、Config 等服務整合，組織層級啟用



AWS Control Tower

- 一鍵建立登陸區域（Landing Zone）：Organizations + Log Archive + Audit 帳戶
- Account Factory：以標準化範本佈建新帳戶
- Account Factory for Terraform（AFT）以 GitOps 方式佈建與客製
- 控制項：預防式（SCP）、偵測式（Config 規則）、主動式（CFN Hooks）
- Customizations for Control Tower（CfCT）套用自訂範本與 SCP
- 儀表板集中檢視合規狀態

標準化並自動化帳戶佈建與設定

1
提出申請
工單或 Git 中的帳戶請求檔

2
建立帳戶
Account Factory / AFT / CreateAccount API

3
放入 OU
自動繼承 SCP 與控制項

4
套用基準
StackSets 自動部署、CfCT 客製

5
啟用安全服務
委派管理員自動啟用 GuardDuty 與 Security Hub

6
授予存取
IAM Identity Center 權限集

帳戶即程式碼

AFT 把帳戶請求、全域與帳戶層級客製都放在 Git，由管道套用

可追蹤、可審查、可重現

基準內容

CloudTrail、Config 錄製、預設 VPC 移除、IAM 角色、預算警示、密碼政策

每個新帳戶一致

多帳戶的 IAM：SCP 與角色切換

服務控制政策 SCP

- 設定帳戶內「最多能有哪些權限」的護欄
- 本身不授予任何權限
- 影響成員帳戶內所有使用者與角色（含 root）
- 不影響管理帳戶與服務連結角色
- 拒絕清單（預設 FullAWSAccess + Deny）或允許清單
- 常見：限制區域、禁止關閉 CloudTrail、禁止離開組織

有效權限 = 所有政策的交集（任何明確 Deny 都優先）

SCP / RCP ∩ 身分政策 ∩ 權限界限 ∩ 工作階段政策 ∩ 資源政策

跨帳戶存取時，資源政策與目標帳戶的身分政策都必須允許

跨帳戶角色切換

1 目標帳戶建立角色

信任政策指定來源帳戶或主體（可加 ExternalId）

2 來源主體獲准呼叫

身分政策允許 sts:AssumeRole 該角色 ARN

3 取得臨時憑證

以角色權限操作目標帳戶資源

大規模實施治理與安全控制



AWS Config

受管 / 自訂規則、一致性套件、彙總器跨帳戶檢視，搭配 SSM Automation 自動修正

偵測式 + 自動修正



AWS Control Tower

預先打包的預防式、偵測式、主動式控制項，依 OU 啟用

一致的護欄



AWS Security Hub

彙整各服務發現項目，依 CIS、AWS 基礎安全最佳實務等標準評分，可跨區域彙總

安全態勢總覽



Amazon GuardDuty

分析 CloudTrail、VPC 流量日誌、DNS 日誌偵測威脅；組織內自動啟用新帳戶

威脅偵測



Amazon Detective

以行為圖譜關聯日誌，調查發現項目的根本原因與影響範圍

事件調查



Service Catalog + SCP

只允許佈建核准的產品；SCP 從上層 OU 阻擋高風險動作

預防式控制

AWS Systems Manager 功能地圖

節點管理

- Fleet Manager
- Inventory 清查
- Session Manager
- Run Command
- State Manager
- Patch Manager
- Distributor

變更管理

- Automation 執行手冊
- Change Manager
- Maintenance Windows
- Change Calendar

應用程式管理

- Parameter Store
- AppConfig
- Application Manager

營運管理

- OpsCenter
- Explorer
- Incident Manager

受管節點的前提：安裝並執行 SSM Agent · 執行個體設定檔含 AmazonSSMManagedInstanceCore · 可連到 Systems Manager 端點（網際網路或 VPC 端點）；本地伺服器以混合啟用註冊

自動化系統清查、組態與修補管理

修補基準

定義哪些修補程式核准安裝（分類、嚴重性、自動核准天數）

修補群組 / 目標

以標籤指定一組執行個體套用某個基準

維護時段

在指定時間執行 AWS-RunPatchBaseline

合規報告

Scan 只回報、Install 才安裝；結果進 Compliance

Patch Manager 流程 · Quick Setup 修補政策可一次套用到整個組織的多帳戶與多區域

Inventory 清查

收集作業系統、應用程式、網路設定、檔案等中繼資料；資源資料同步到 S3，用 Athena / QuickSight 查詢

跨帳戶集中清查

AWS Config

記錄 AWS 資源組態變更歷史與關聯；可回答「誰在何時改了什麼設定」

資源層級清查

以 Lambda、SDK 與 Step Functions 自動化複雜情境



事件來源

EventBridge 規則（AWS API 事件、排程）、S3、SNS、CloudWatch 警示



Lambda + SDK

以 boto3 等 AWS SDK 呼叫 API 執行單一步驟



Step Functions

編排多個步驟：重試、錯誤處理、平行、等待、人工核准



結果與通知

SNS、Slack、OpsCenter 工作項目、寫回資料庫

何時從 Lambda 升級到 Step Functions

- 流程超過 Lambda 15 分鐘上限
- 需要重試、退避、錯誤分支等可視化流程控制
- 需等待人工核准（任務權杖回呼 .waitForTaskToken）
- 大量平行處理（Map 狀態）；可直接呼叫 200 多種 AWS 服務 API

範例：自動處理不合規的安全群組

- EventBridge 捕捉 AuthorizeSecurityGroupIngress 事件
- Lambda 檢查是否開放 0.0.0.0/0 的 SSH
- 不合規 → 以 SDK 撤銷規則
- SNS 通知擁有者並記錄到 OpsCenter

把軟體組態自動維持在期望狀態

Run Command

一次性

- 對大量節點立即執行指令或文件
- 以標籤或資源群組選取目標
- 控制速率與錯誤閾值
- 不會持續維持狀態

State Manager

持續維持

- 關聯（Association）= 文件 + 目標 + 排程
- 定期重新套用，修正組態漂移
- 新加入的執行個體自動套用
- 可執行 Ansible、Chef、PowerShell DSC

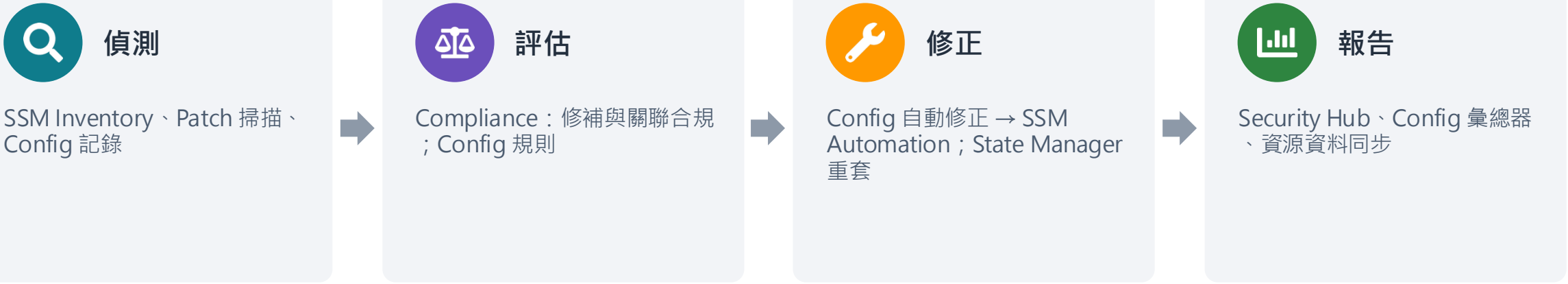
Automation

多步驟執行手冊

- 呼叫 AWS API、執行腳本、核准步驟
- 例：AWS-UpdateLinuxAmi 建立修補後 AMI
- 可跨帳戶與區域執行
- 作為 Config 自動修正的動作

OpsWorks（Chef / Puppet）已終止服務：考綱仍以它為例，現行替代方案是 State Manager 搭配 AWS-ApplyChefRecipes 或 AWS-ApplyAnsiblePlaybooks 等文件，維持期望狀態

維持軟體合規



合規需求	做法
所有執行個體必須安裝最新安全修補	Patch Manager 基準 + 維護時段；Config 規則 ec2-managedinstance-patch-compliance-status-check
禁止安裝未授權軟體	Inventory 收集應用程式清單 + 自訂 Config 規則或 Lambda 檢查，偵測後以 Run Command 移除
只能使用核准的 AMI	Config 受管規則 approved-amis-by-id / by-tag；Image Builder 產出核准 AMI
必須安裝並執行監控代理程式	State Manager 關聯持續安裝 CloudWatch Agent（AWS-ConfigureAWSPackage）

情境練習

1. EC2 啟動腳本完成前堆疊已顯示完成 [redacted]

2. 新帳戶加入 OU 時自動部署 Config [redacted]

3. 使用者自助佈建核准範本、不給資源權限 [redacted]

4. 不重新部署就切換功能旗標並可回滾 [redacted]

5. 禁止所有帳戶在未核准區域建立資源 [redacted]

6. 快速建立符合最佳實務的多帳戶環境 [redacted]

7. 調查 GuardDuty 發現項目的根本原因 [redacted]

8. 每週日凌晨自動為所有 EC2 安裝修補 [redacted]

9. 持續確保所有執行個體安裝監控代理程式 [redacted]

10. 需人工核准且超過 15 分鐘的自動化流程 [redacted]

情境練習

對照你的答案，再回頭複習對應任務

1. EC2 啟動腳本完成前堆疊已顯示完成 — **CreationPolicy + cfn-signal**

2. 新帳戶加入 OU 時自動部署 Config — **StackSets 自動部署**

3. 使用者自助佈建核准範本、不給資源權限 — **Service Catalog 啟動限制**

4. 不重新部署就切換功能旗標並可回滾 — **AWS AppConfig**

5. 禁止所有帳戶在未核准區域建立資源 — **SCP 限制區域**

6. 快速建立符合最佳實務的多帳戶環境 — **AWS Control Tower**

7. 調查 GuardDuty 發現項目的根本原因 — **Amazon Detective**

8. 每週日凌晨自動為所有 EC2 安裝修補 — **Patch Manager + 維護時段**

9. 持續確保所有執行個體安裝監控代理程式 — **State Manager 關聯**

10. 需人工核准且超過 15 分鐘的自動化流程 — **Step Functions**

自我檢核：領域 2 官方目標

2.1	比較 AWS 上的 IaC 選項	CloudFormation、SAM、CDK、Terraform、CLI / SDK
2.1	IaC 變更管理流程	變更集、堆疊政策、漂移偵測、回滾觸發條件
2.1	撰寫與部署 IaC 範本	範本區段、cfn-signal、DeletionPolicy、cdk bootstrap
2.1	跨帳戶 / 區域套用 StackSets	自行管理 vs 服務管理權限、自動部署
2.1	選擇組態管理服務	Systems Manager、Config、AppConfig、OpsWorks
2.1	可重複使用且具治理的 IaC	Service Catalog、CFN 模組、CDK Construct、Hooks
2.2	帳戶結構與佈建自動化	OU 設計、Account Factory、AFT、基準設定
2.2	集中管理帳戶	Organizations、Control Tower、委派管理員
2.2	多帳戶 IAM	SCP、有效權限交集、跨帳戶角色、Identity Center
2.2	大規模治理與安全控制	Config、Security Hub、GuardDuty、Detective
2.3	清查、組態與修補自動化	Inventory、Patch Manager、維護時段
2.3	Lambda / Step Functions 自動化	EventBridge、SDK、重試、人工核准
2.3	期望狀態與軟體合規	Run Command、State Manager、Automation、Compliance